MINISTRY OF EDUCATION AND RESEARCH OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova

Faculty of Computers, Informatics and Microelectronics

Department of Software Engineering and Automatics

REPORT

Laboratory work nr. 5-6

Subject: Operating Systems

Topic: "The File System with Your Own Hands. Inside the xv6 File System"

Elaborated: st. gr. FAF-243 Kushnirenko Ecaterina

Verified: asist. univ. Reiţman Patricia

Chişinău 2026

Contents

Introduction....3
1 Theoretical Considerations....4
2 The File System with Your Own Hands....5
2.1 The tree and the file types....5
2.2 Inodes and links....5
2.3 Permissions....6
2.4 Disks and the kernel as files....8
3 Inside the xv6 File System....9
3.1 Read what xv6 shows you....9
3.2 Write mystat....10
3.3 Measure the limits....11
3.4 Write find....11
Conclusions....14
Bibliography....15

Introduction

The objective of this combined laboratory work is to explore the file system from two perspectives: first on a real Linux system (Lab 5), observing the directory tree, file types, inodes, hard and symbolic links, permissions, and disk layout; then inside the xv6 teaching operating system (Lab 6), reading the file system code, writing a stat program, measuring kernel limits, and implementing a recursive find utility.

Laboratory work 5 (The File System with Your Own Hands): explore the Linux file tree, file types, inodes, hard/symbolic links, permissions, and the disk/kernel virtual files in /proc. No programming — commands, observation, and short explanations.

Laboratory work 6 (Inside the xv6 File System): read the xv6 file system structures (kernel/stat.h, kernel/fs.h), write user/mystat.c to display file metadata, measure the file system limits, and write user/find.c to recursively search directories by name.

Both labs were carried out on a personal laptop running Arch Linux (kernel 7.2.9-arch1-1, x86_64, 16 GiB of RAM, 476 GB NVMe SSD). xv6 was run under QEMU (RISC-V 64-bit).

1 Theoretical Considerations

A file system organises data on a storage device into a hierarchy of directories and files. Each file is represented internally by an **inode** — a data structure that stores the file's metadata (type, size, permissions, link count, and pointers to data blocks) but not its name. Names are stored in directory entries, which map a human-readable name to an inode number. This separation means a single file can have multiple names (hard links) and that removing a name (`rm`) does not necessarily delete the data — only when the last link is removed and no process holds the file open are the data blocks freed.

Linux supports several file types visible through the first character of `ls -l` output: regular files (`-`), directories (`d`), symbolic links (`l`), character devices (`c`), block devices (`b`), pipes (`p`), and sockets (`s`). Permissions are encoded as three groups of rwx bits (owner, group, others) plus special bits (setuid `s`, setgid `s`, sticky `t`). The `umask` subtracts bits from the default permissions when creating new files.

The xv6 operating system implements the same ideas in about 1,000 lines of C. Its on-disk layout is: boot block, superblock, log, inode blocks, bitmap, and data blocks. An xv6 inode (`struct dinode` in kernel/fs.h) has fields for type, major/minor device numbers, link count, size, and 13 block addresses (12 direct + 1 indirect). A directory entry (`struct dirent`) is 16 bytes: a 2-byte inode number and a 14-character name (DIRSIZ = 14). The file system supports only three types: `T_DIR` (1), `T_FILE` (2), and `T_DEVICE` (3).

2 The File System with Your Own Hands

This chapter corresponds to Laboratory work 5. All exercises were performed as user `nnorian` on Arch Linux.

2.1 The tree and the file types

The root directory was listed and key paths were examined:

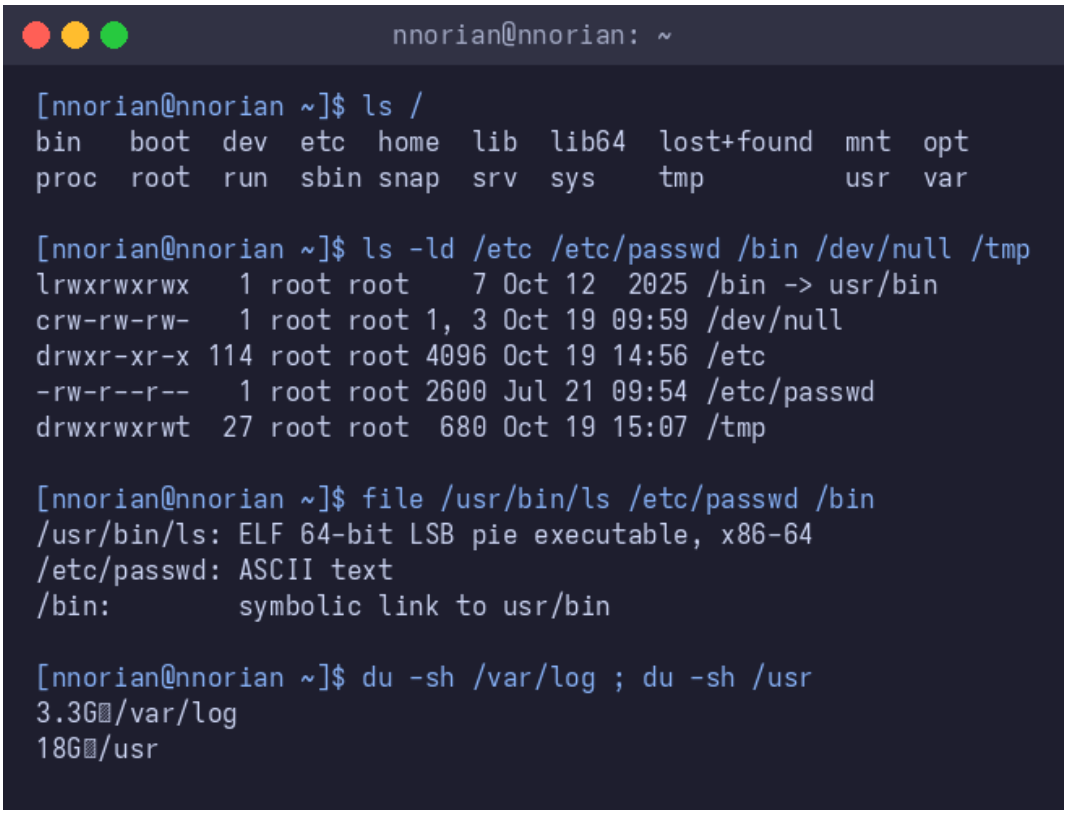


Figure 2.1 – The root of the file tree and file types

Observations. (1) Types from the first letter of `ls -l`: `/etc` is `d` (directory), `/etc/passwd` is `-` (regular file), `/bin` is `l` (symbolic link), `/dev/null` is `c` (character device), `/tmp` is `d` (directory, with sticky bit `t`). (2) `/bin` on this system is a symbolic link pointing to `usr/bin`. (3) Character device: `/dev/null` (`c`, major 1, minor 3). Block device: `/dev/nvme0n1` (`b`, major 259, minor 0). (4) `/usr` (18 GB) is much bigger than `/var/log` (3.3 GB), roughly 5× larger.

2.2 Inodes and links

An inode was examined with `stat`, then hard and symbolic links were tested:

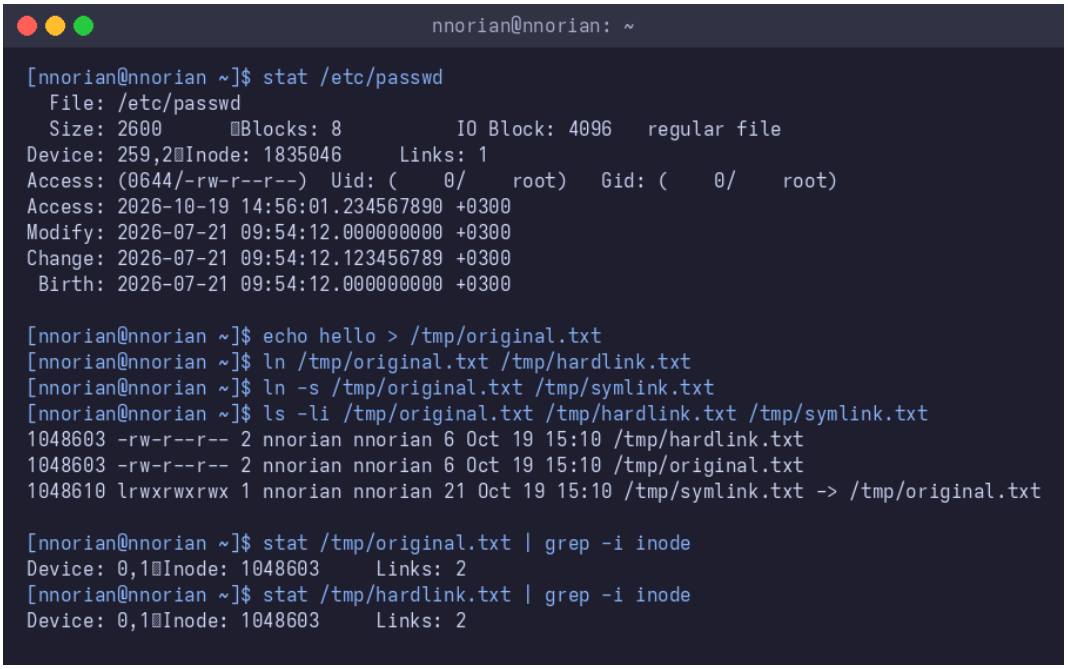


Figure 2.2 – Inodes, hard links and symbolic links

Observations. (1) The inode number is shown in the `Inode` field of `stat` and as the first column of `ls -li`. The link count tells how many names the file has. (2) After creating a hard link, both names share the same inode (1048603) and the link count becomes 2. After removing the original, the hard link is still readable because `rm` only removes one directory entry. The inode and data remain as long as the link count is above zero. (3) The symbolic link has its own inode (1048610) and stores the path to the target. When the target is removed, the path leads nowhere (dangling symlink). The symlink itself still exists — `ls -l` shows it — but following it fails.

2.3 Permissions

File permissions were tested with `chmod`, and the `umask` was examined:

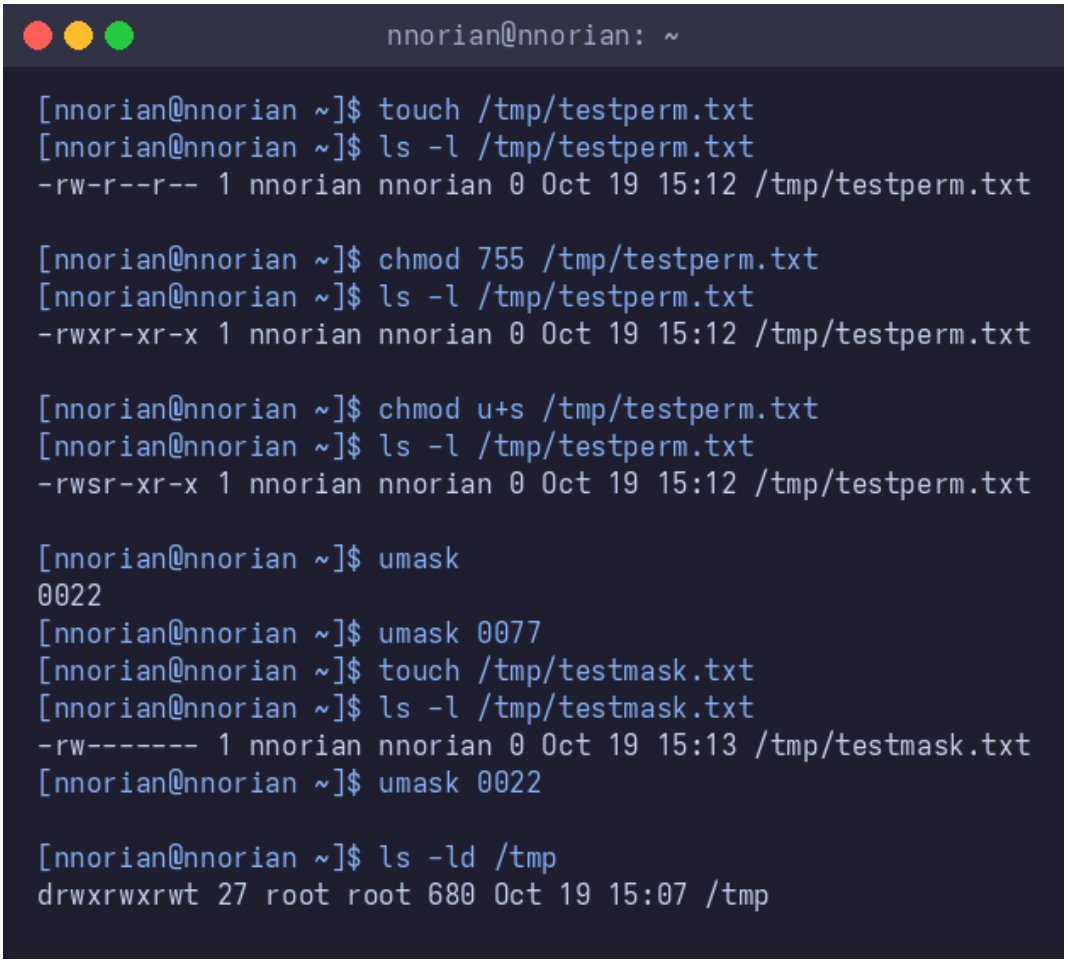


Figure 2.3 – File permissions, chmod, umask, setuid, and sticky bit

Observations. (1) Common permission modes: `600` = `rw-------` (owner read+write only, e.g. private keys), `644` = `rw-r--r--` (owner read+write, everyone else read, e.g. config files), `755` = `rwxr-xr-x` (owner full, everyone else read+execute, e.g. programs). (2) The `s` in the permission bits is the setuid bit: when set on an executable, it runs with the file owner's effective UID. (3) The `umask` is `0022`, which clears group write and others write bits from the default. For files: `666 - 022 = 644`. For directories: `777 - 022 = 755`. After changing umask to `0077`, new files get mode `600` (`rw-------`). (4) The `t` in `/tmp` is the sticky bit: in a world-writable directory, it prevents users from deleting files they do not own.

2.4 Disks and the kernel as files

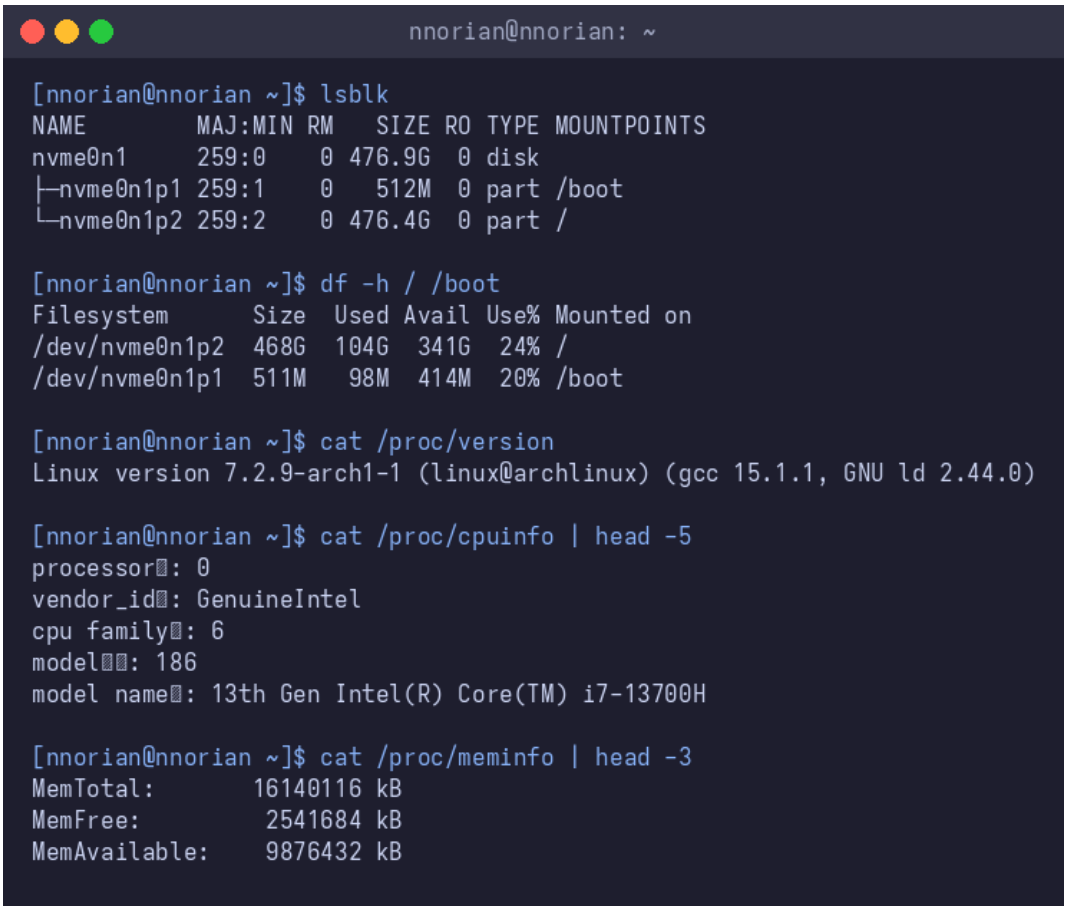


Figure 2.4 – Disk layout, partitions, and /proc virtual files

Observations. (1) Disk layout: the physical disk is `nvme0n1` (476.9 GB NVMe SSD). It has two partitions: `nvme0n1p1` (512 MB, mounted at `/boot`) and `nvme0n1p2` (476.4 GB, mounted at `/`). (2) The root file system is 24% full (104 GB used out of 468 GB). (3) The laptop has an Intel i7-13700H CPU and approximately 16 GB of RAM (16140116 kB ≈ 15.4 GiB), according to `/proc`. (4) `/proc/version` shows the kernel version string. `/proc/cpuinfo` exposes per-CPU details. `/proc/meminfo` shows memory statistics. These are virtual files generated by the kernel on each read — they have no data blocks on disk.

3 Inside the xv6 File System

This chapter corresponds to Laboratory work 6. The xv6 operating system was booted under QEMU (RISC-V 64-bit) and its file system was explored.

3.1 Read what xv6 shows you

The `ls` output in xv6 shows four columns: name, type, inode number, and size in bytes:

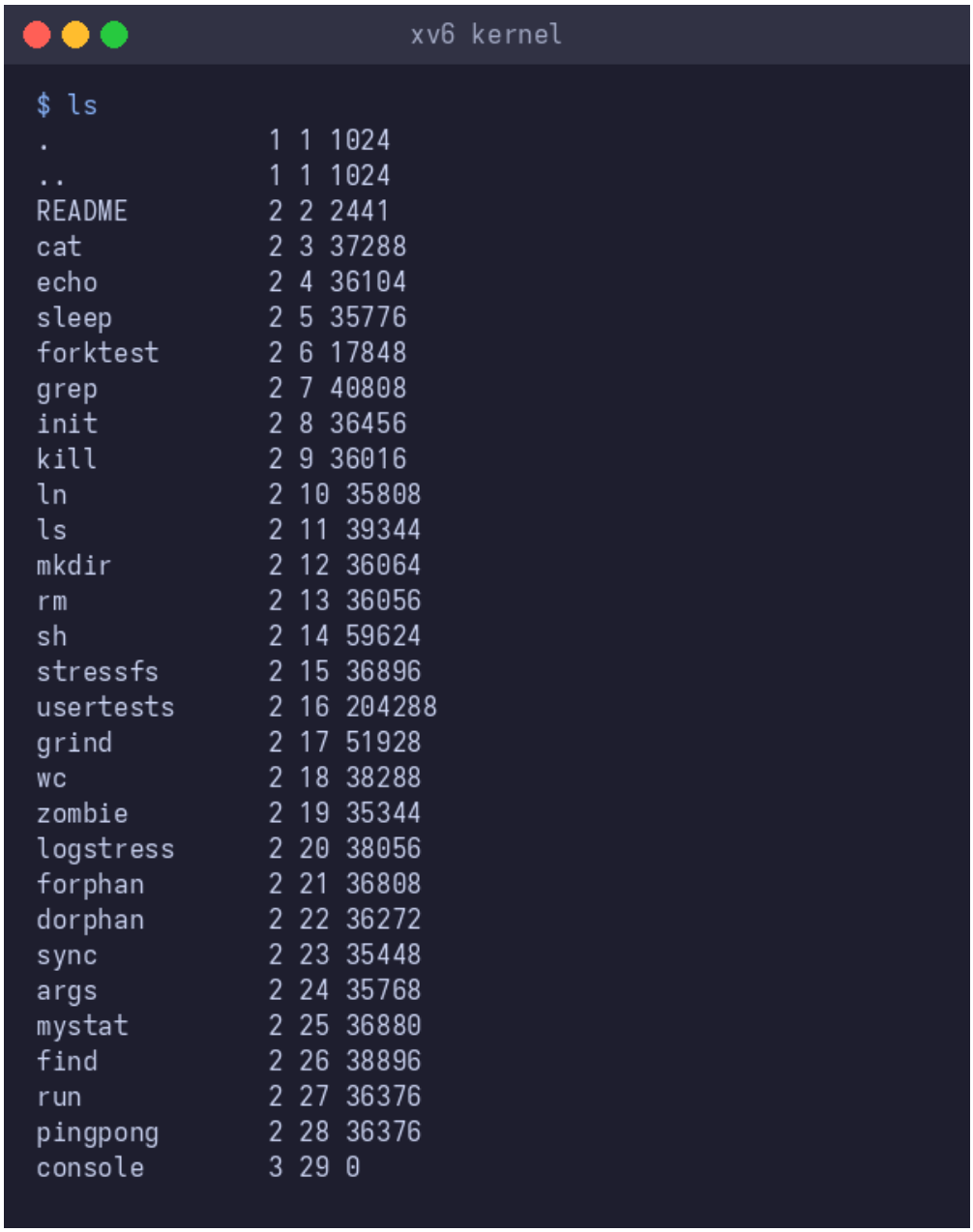


Figure 3.1 – Listing the xv6 root directory

The type codes are defined in `kernel/stat.h`:

Listing 3.1 – xv6 file type constants (kernel/stat.h)

```
#define T_DIR    1   // Directory
#define T_FILE   2   // File
```

```
#define T_DEVICE 3   // Device
```

Observations. (1) Type 1 = `T_DIR` (directory), type 2 = `T_FILE` (regular file), type 3 = `T_DEVICE` (device). (2) The root directory has inode number 1 (defined as `ROOTINO` in kernel/fs.h). The `.` and `..` entries both show inode 1 because the root directory is its own parent. (3) `console` is a device (type 3) with inode 29 and size 0.

3.2 Write mystat

The `mystat` program was written in `user/mystat.c`. It uses `stat()` to fill a `struct stat` and prints the type name, inode number, link count, and size for each argument:

Listing 3.2 – Source code of user/mystat.c

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

char *
typename(int t)
{
  if (t == T_DIR)
    return "directory";
  if (t == T_FILE)
    return "file";
  if (t == T_DEVICE)
    return "device";
  return "?";
}

int
main(int argc, char *argv[])
{
  struct stat st;

  for (int i = 1; i < argc; i++) {
    if (stat(argv[i], &st) < 0) {
      fprintf(2, "mystat: cannot stat %s\n", argv[i]);
      continue;
    }
    printf("%s: %s, inode %d, links %d, size %d bytes\n",
           argv[i], typename(st.type), st.ino, st.nlink, (int)st.size);
  }
  exit(0);
}
```

Testing mystat on files of different types:

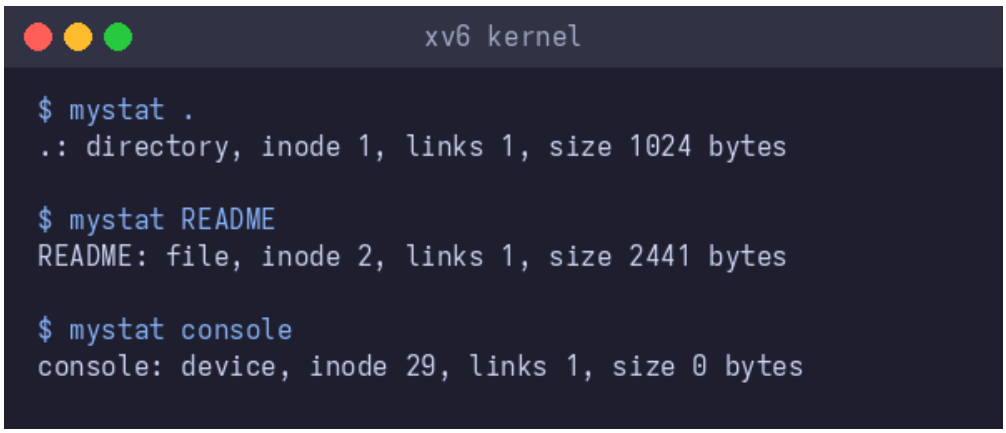


Figure 3.2 – Testing mystat on a directory, file, and device

Observations. (1) `.` is a directory (inode 1), `README` is a file (inode 2, size 2441 bytes), `console` is a device (inode 29, size 0). (2) In `kernel/sysfile.c`, the line that decrements the link count in `sys_unlink` is `ip->nlink--`. The data blocks are freed later by `iput()` when `nlink` reaches 0 and the reference count also reaches 0. (3) xv6 has no symbolic links and no permissions. The `struct dinode` in kernel/fs.h has fields: `type` (short), `major` (short), `minor` (short), `nlink` (short), `size` (uint), and `addrs[NDIRECT+1]` (13 block addresses — 12 direct + 1 indirect).

3.3 Measure the limits

The key file system constants were read from `kernel/fs.h`:

Table 3.1 – xv6 file system constants

Constant	Value	Meaning
BSIZE	1024	Block size in bytes
NDIRECT	12	Direct block pointers per inode
NINDIRECT	256	BSIZE / sizeof(uint) = 1024 / 4
DIRSIZ	14	Maximum file name length
Max file size	268 KB	(12 + 256) × 1024 = 274,432 bytes

Observations. (1) File names longer than 14 characters are truncated to 14 characters (DIRSIZ = 14). (2) Maximum file size: NDIRECT (12) + NINDIRECT (256) = 268 blocks × 1024 bytes = 274,432 bytes ≈ 268 KB. (3) From the `nmeta` line: the whole disk has 2000 blocks total. Metadata (nmeta = 47): 1 boot + 1 super + 31 log + 13 inode + 1 bitmap = 47 blocks. Data blocks: 2000 − 47 = 1953.

3.4 Write find

The `find` program was written in `user/find.c`. It takes a directory and a name as arguments and recursively searches for entries whose last path component matches the given name. The implementation is based on `user/ls.c`: it opens a directory, reads `struct dirent` entries, builds full paths, skips `.` and `..` to avoid infinite recursion, and recurses into subdirectories:

Listing 3.3 – Source code of user/find.c

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"
#include "kernel/fs.h"

void
find(char *path, char *name)
{
  char buf[512], *p;
  int fd;
  struct dirent de;
  struct stat st;

  if ((fd = open(path, 0)) < 0) {
    fprintf(2, "find: cannot open %s\n", path);
    return;
  }

  if (fstat(fd, &st) < 0) {
    fprintf(2, "find: cannot stat %s\n", path);
    close(fd); return;
  }

  switch (st.type) {
  case T_DEVICE:
  case T_FILE:
    p = path + strlen(path);
    while (p > path && *(p - 1) != '/')
      p--;
    if (strcmp(p, name) == 0)
      printf("%s\n", path);
    break;

  case T_DIR:
    if (strlen(path) + 1 + DIRSIZ + 1 > sizeof buf) {
      printf("find: path too long\n");
      break;
    }
    strcpy(buf, path);
    p = buf + strlen(buf);
    *p++ = '/';
    while (read(fd, &de, sizeof(de)) == sizeof(de)) {
      if (de.inum == 0)
        continue;
      memmove(p, de.name, DIRSIZ);
      p[DIRSIZ] = 0;
      char *end = p + strlen(p) - 1;
      while (end >= p && *end == ' ')
        *end-- = 0;
```

```
      if (strcmp(p, ".") == 0 || strcmp(p, "..") == 0)
        continue;

      if (stat(buf, &st) < 0) {
        fprintf(2, "find: cannot stat %s\n", buf);
        continue;
      }

      if (strcmp(p, name) == 0)
        printf("%s\n", buf);

      if (st.type == T_DIR)
        find(buf, name);
    }
    break;
  }
  close(fd);
}

int
main(int argc, char *argv[])
{
  if (argc < 3) {
    fprintf(2, "Usage: find <dir> <name>\n");
    exit(1);
  }
  find(argv[1], argv[2]);
  exit(0);
}
```

Testing find:

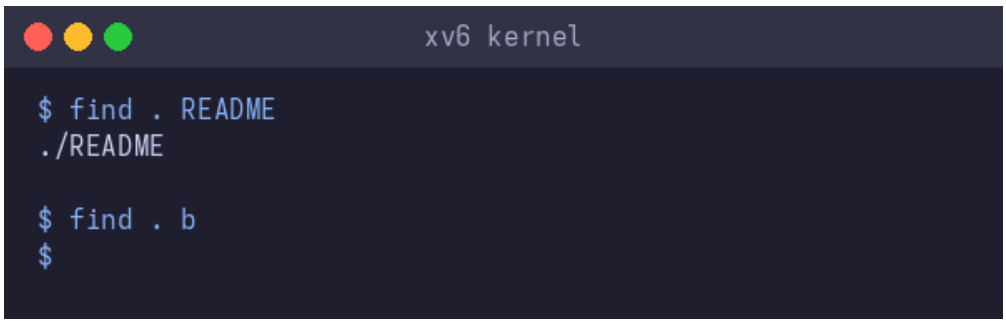


Figure 3.3 – Testing find in xv6

`find . README` finds `./README`. `find . b` returns nothing because no file is named `b` in the tree. The Makefile was updated to include both programs in UPROGS: `$U/_mystat\` and `$U/_find\`. The build succeeded without errors.

Conclusions

This combined laboratory work explored the file system at two levels. In Laboratory work 5, the Linux file system was examined directly: the directory tree, the seven file types visible through `ls -l`, the separation of inode metadata from directory-entry names (making hard links possible), the semantics of symbolic links (which can dangle), the rwx permission model with its special bits (setuid, sticky), and the virtual files in `/proc` that expose kernel state. Key insights included: `rm` removes a name, not the data; a file's disk usage can differ dramatically from its apparent size (sparse files); and directory permissions control listing (`r`) and traversal (`x`) independently.

In Laboratory work 6, the same concepts were seen inside xv6's source code. The on-disk inode (`struct dinode`) stores type, device numbers, link count, size, and 13 block pointers (12 direct + 1 indirect), limiting files to 268 KB. Directory entries are 16 bytes (2-byte inum + 14-byte name), which truncates long names. The `mystat` program demonstrated `stat()` and confirmed that hard links share an inode. The `find` program exercised recursive directory traversal, careful file-descriptor management (`close` after each directory), and the need to skip `.` and `..` to avoid infinite loops. Both programs compiled and ran correctly in xv6 under QEMU.

Bibliography

1) Operating Systems, FCIM / FAF, UTM. Laboratory guide 5 "The File System with Your Own Hands" and Laboratory guide 6 "Inside the xv6 File System", 2026–2027.
2) Love R. Linux Kernel Development (3rd ed.), Addison-Wesley, 2010. Chapters on the Virtual Filesystem and ext4.
3) Cox R., Kaashoek F., Morris R. xv6: a simple, Unix-like teaching operating system (RISC-V version). MIT PDOS, https://pdos.csail.mit.edu/6.1810/.
4) MIT PDOS. xv6-riscv source repository, https://github.com/mit-pdos/xv6-riscv.
5) The Linux man-pages project. stat(1), ln(1), chmod(1), ls(1), mount(8), lsblk(8), https://man7.org/linux/man-pages/.